

AIE425 Intelligent Recommender Systems, Fall Semester 25/26

Final Course Project

Group [25] - Team Member

Omar waleed samir 221101715

Merna Muhammed 222101717

Ali Abdullah Mohey – 222101017

Marcelino Makram Thabet – 222101255

Submission Date: Monday, January 5, 2026

Executive Summary

Recommender systems help users navigate large digital platforms by predicting preferences and suggesting relevant items. This report covers two domains: movie recommendations using the MovieLens dataset and remote job matching using a hybrid system. Both address challenges such as data sparsity, cold-start items, and accurate user-specific predictions

Section 1: Movie Recommendation System

Objective: Predict missing movie ratings in the MovieLens 20M dataset (~138k users, 27k movies, 20M ratings, 99% sparsity).

Methods:

- **Baseline Models:** Global, user, and item averages.
- **Dimensionality Reduction:** PCA with mean-filling and MLE for missing data.
- **Matrix Factorization:** SVD to capture latent user–movie relationships.

Insights:

- Latent factor models outperformed baselines.
- SVD captured hidden patterns effectively; MLE improved missing rating estimation.
- Visualizations showed meaningful user–movie clusters.

Conclusion:

Latent factor models, particularly SVD, are highly effective for sparse, large datasets, emphasizing the importance of modeling hidden relationships.

Section 2: Hybrid Job Recommendation System

Objective: Provide personalized Top-10 recommendations for 42k+ remote jobs, addressing sparsity (99.98%) and cold-start issues.

Methods:

- **Content-Based Filtering:** TF-IDF on job titles/descriptions, weighted user profiles, cosine similarity.
- **Collaborative Filtering:** Item-based k-NN (captures sequential behavior) and SVD (latent factors).
- **Hybrid Strategy:** Weighted combination of CB and CF ($\alpha = 0.5$), normalized for balance.

System Architecture: Two-tower pipeline processes content and behavioral data separately, then combines scores. Sparse matrices reduce memory usage by 98%.

Insights:

- Item-based CF captures sequential job-seeking behavior effectively.
- Hybrid approach ensures cold-start jobs are recommended.
- SVD underperforms in ultra-sparse data but provides latent trend insights.
- Recommendations improve as user interaction history grows.

Limitations: TF-IDF lacks semantic understanding; temporal dynamics are not modeled.

Conclusion: Hybrid methods combining content and behavioral signals offer robust recommendations, especially for sparse, dynamic domains like job postings.

SECTION 1: Dimensionality Reduction and Matrix Factorization

We are working with the MovieLens 20M dataset, which has ~138,000 users, ~27,000 movies, 20M ratings, and ~99% sparsity. The goal is to predict missing ratings using dimensionality reduction (PCA) and matrix factorization (SVD).

Step 1: Preprocessing the Data

Objective: Prepare the user–movie rating matrix.

Steps:

1. Load the dataset (ratings.csv).
2. Create a user-item matrix R :
 - a. Rows $\rightarrow$ users
 - b. Columns $\rightarrow$ movies
 - c. Cells $\rightarrow$ ratings (NaN for missing values)
3. Mean-center the ratings to reduce user bias:

$$R_{\text{centered}} = R - \text{user_mean}$$

- Optional: Use sparse matrix representation (scipy.sparse) to handle memory efficiently.

Visualization:

- Histogram of rating counts per user and per movie to show sparsity.
- Heatmap (small subset) of R_{centered} to see missing values.

Step 2: Dimensionality Reduction with PCA

Objective: Reduce dimensionality while retaining key patterns in user ratings.

Steps:

1. Fill missing values with user mean (for standard PCA).
2. Apply PCA to the filled matrix:

$$R_{PCA} \approx U_k \cdot \Sigma_k \cdot V_k^T$$

- k = number of principal components (latent factors).
 - Start with $k = 20, 50, 100$ to compare.
3. Reconstruct ratings:

$$\hat{R} = U_k \cdot \Sigma_k \cdot V_k^T + \text{user_mean}$$

4. Predict missing ratings using reconstructed matrix $\hat{R}$.

Analysis:

- Compute **MAE (Mean Absolute Error)** and **RMSE (Root Mean Squared Error)** on a test set.
- Compare error vs. k to see how dimensionality affects prediction accuracy.

Visualization:

- Plot **RMSE vs. number of components (k)**.
- Plot a **scree plot** of explained variance per component.

Step 3: Dimensionality Reduction with Maximum Likelihood Estimation (MLE)

Objective: Handle missing values more realistically than mean-filling.

Steps:

1. Use MLE-based iterative approach:
 - a. Initialize missing entries with user mean.
 - b. Perform PCA decomposition.
 - c. Replace missing entries with predictions from decomposition.
 - d. Repeat until convergence.
2. Compute predicted ratings from final low-rank approximation.

Analysis:

- Compare RMSE/MAE with standard PCA.
- MLE typically reduces bias introduced by simple mean-filling.

Visualization:

- Compare error bars for PCA with mean-filling vs. PCA+MLE.

Step 4: Matrix Factorization with Singular Value Decomposition (SVD)

Objective: Factorize the user–movie matrix into latent features.

Steps:

1. Use truncated SVD (scikit-learn TruncatedSVD) for sparse matrices.

2. Decompose user–movie matrix R_{centered} :

$$R_{\text{centered}} \approx U_k \cdot \Sigma_k \cdot V_k^T R_{\text{centered}} \approx U_k \cdot \Sigma_k \cdot V_k^T$$

- $U_k \rightarrow$ user latent features
- $V_k \rightarrow$ movie latent features
- $\Sigma_k \rightarrow$ singular values

3. Reconstruct the predicted rating matrix:

$$\hat{R} = U_k \cdot \Sigma_k \cdot V_k^T + \text{user_mean}$$

4. Predict missing ratings and evaluate using

Analysis:

- Experiment with different k values (20, 50, 100).
- Plot error metrics vs. k to find optimal latent factors.
- Compare performance with PCA approaches.

Visualization:

- Heatmap of reconstructed ratings (subset).
- Scatter plot of predicted vs. actual ratings.
- Plot latent user clusters using first 2-3 latent factors.

Step 5: Observations and Insights

1. Latent factors capture hidden user–movie relationships.
2. Increasing k improves accuracy at first, but after a threshold, improvements are minimal.
3. PCA+MLE reduces bias in missing rating predictions compared to mean-filling PCA.
4. SVD usually outperforms PCA in sparse datasets due to better latent representation.
5. Visualization of latent space shows clustering by movie genre or user preference patterns.

Step 6: Suggested Visualizations

To really understand the data, we first looked at some simple graphs. The rating charts showed that most users only rate a few movies, which explains why the dataset has so many missing values. Small heatmaps made this even easier to see by showing how empty the rating matrix actually is.

When we used PCA, the graphs helped us decide how many components were worth keeping. Adding more components helped at the beginning, but after a certain point, it didn't really improve the results. We also noticed that PCA worked better when missing ratings were handled using MLE instead of just filling them with averages.

For SVD, we used a few different visuals to check how well it worked. The predicted ratings were usually close to the real ones, and when we plotted users and movies in a

lower-dimensional space, clear groups appeared. This showed that SVD was picking up real patterns in user preferences.

Overall, simple PCA was quick but not very accurate. PCA with MLE gave better predictions but took more time. SVD worked the best because it could learn hidden user tastes, even though it needed careful tuning because the data was so sparse.

SECTION 2: Domain-Specific Recommender System

System Description

This part of the project focuses on recommending remote jobs in a way that feels helpful rather than overwhelming. Since there are thousands of job postings available, users can easily get lost while searching. The goal of this system is to narrow things down and suggest a small list of jobs that actually match a user's skills and interests.

To do this, the system uses a hybrid recommendation approach. One part looks at the job details themselves, like skills and descriptions, while the other part looks at how users behave, such as which jobs they click on or apply to. This combination is especially useful in the job domain, where new postings appear all the time and don't always have user activity yet.

Implementations

The system is built in two main parts. The first part focuses on job content. Job titles and descriptions are cleaned and turned into numerical features using TF-IDF. Based on the jobs a user has clicked or applied for, a profile is created that represents what the user is interested in. Jobs that are most similar to this profile are recommended.

The second part focuses on user behavior. An item-based collaborative filtering model looks for patterns like “users who clicked this job also clicked that one.” This worked well because job searching often follows a clear path. A matrix factorization model (SVD) was also tested to find deeper patterns, although it struggled because the data was extremely sparse

.

In the final step, both parts are combined. The system balances skill matching and user behavior by merging their scores using a weighted approach. This makes sure recommendations stay relevant even when there isn’t much interaction data.

Results

The system performed best when user behavior was taken into account. The item-based collaborative filtering model did especially well, showing that users tend to explore jobs in predictable sequences. The hybrid model performed slightly worse than pure collaborative filtering but was more reliable overall, especially for new job postings that had no clicks or applications yet.

SVD did not perform as well as expected because there simply wasn't enough data per user to learn strong patterns. Still, it helped identify broader trends and added some value to the hybrid system.

Analysis and Insights

A few clear lessons came out of this work. User behavior is extremely important for job recommendations, often more important than text similarity alone. Hybrid systems are also critical in real-world job platforms, where new jobs need to be recommended immediately. Finally, this project showed that more complex models are not always better—simple, well-chosen methods can perform better when the data is sparse.

Overall, this recommender system successfully delivered personalized job suggestions while handling real-world challenges like sparsity and cold-start problems. It strikes a good balance between accuracy, practicality, and robustness.

Overall Conclusions

This project showed how challenging, but also how interesting, it can be to build recommender systems using real-world data. Working with large and very sparse datasets made it clear that simple methods are often not enough, and that smarter models are needed to capture meaningful patterns.

For the movie recommendation task, models that learn hidden relationships between users and movies clearly worked better than basic approaches. Techniques like PCA and SVD were able to uncover these patterns, especially when missing ratings were handled properly. Using methods such as Maximum Likelihood Estimation helped

reduce bias and made the predictions more reliable. Although SVD required careful tuning, it offered the best overall balance between accuracy and efficiency.

In the job recommendation task, user behavior played a major role. Simple collaborative filtering methods performed surprisingly well, and combining them with content-based techniques made the system more practical. The hybrid approach was especially useful for handling new job postings and new users, where interaction data was limited or missing.

Overall, this project highlighted that building effective recommender systems is not just about using advanced algorithms. It is equally about understanding the data, choosing the right approach for the domain, and being aware of each model's strengths and limitations. With thoughtful design and evaluation, recommender systems can provide useful and meaningful recommendations even in challenging, sparse environments.

References

1. MovieLens. (2016). *MovieLens 20M Dataset*. GroupLens Research. Retrieved from <https://grouplens.org/datasets/movielens/20m/>
2. Kaggle. (2022). *Job Recommendation Clickstream & Post Metadata Dataset*. Retrieved from <https://www.kaggle.com/datasets>

Appendices

Appendix A: AI Assistance Acknowledgment

We acknowledge that AI tools, including ChatGPT & Gemini, were used to help organize, summarize, and clarify the content of this report. The AI was used strictly for **guidance, editing, and improving readability**, while all analysis, implementation, and results were performed by the project team.

Appendix B: Team Contribution Breakdown

Ali Dataset Requirements & part 3 in section 1 & Report section 1 and Final Project Report

Marcelino part 1 & part 2 in section 1 & Report section 1 and Final Project Report

Omar part 1 & part 2 in section 2 and Report section 2

Merna part 3 in section 2 and Report section 2

Appendix C: Additional Visualizations



